COMPUTER ARCHITECTURE

Computer architecture refers to the arrangement of the various components into a computer—not to whether the box has Doric or Ionic columns or a custom shade of beige like the one American entrepreneur Steve Jobs (1955–2011) created for the original Macintosh computer. Many different architectures have been tried over the years. What's worked and what hasn't makes for fascinating reading, and many books have been published on the subject.

Basic Architectural Elements

The two most common architectures are the *von Neumann* (named after Hungarian-American wizard John von Neumann, 1903–1957) and the *Harvard* (named after the Harvard Mark I computer, which was, of course, a Harvard architecture machine). We've already seen the parts; Figure 5-1 shows how they're organized.

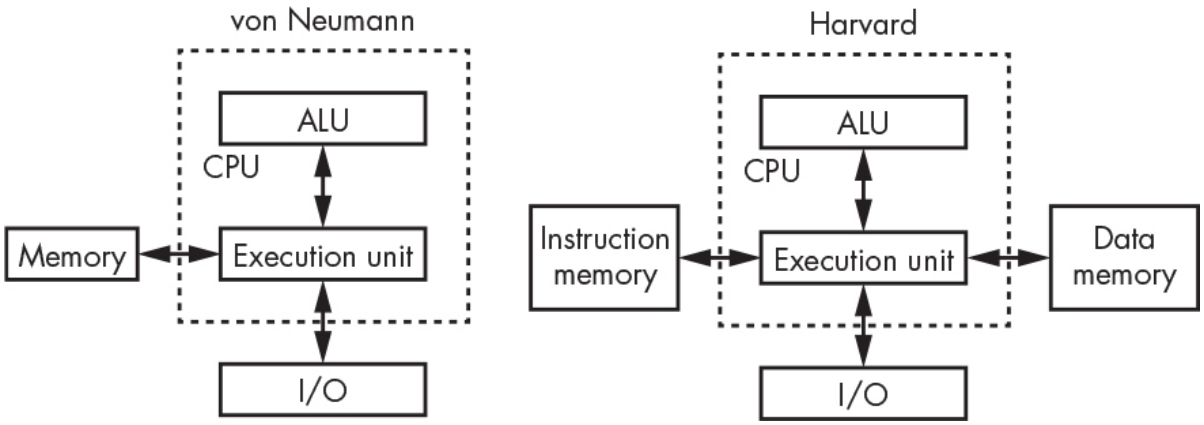

Figure 5-1: The von Neumann and Harvard architectures

Notice that the only difference between them is the way the memory is arranged. All else being equal, the von Neumann architecture is slightly slower because it can't access instructions and data at the same time, since there's only one memory bus. The Harvard architecture gets around that but requires additional hardware for the second memory bus.

Processor Cores

Both architectures in Figure 5-1 have a single CPU, which, as we saw in Chapter 4, is the combination of the ALU, registers, and execution unit. *Multiprocessor* systems with multiple CPUs debuted in the 1980s as a way to get higher performance than could be achieved with a single CPU. As it turns out, though, it's not that easy. Dividing up a single program so that it can be *parallelized* to make use of multiple CPUs is an unsolved problem in the general case, although it works well for some things such as particular types of heavy math. However, it's useful when you're running more than one program at the same time and was a lifesaver in the early days of graphical workstations, as the X Window System was such a resource hog that it helped to have a separate processor to run it.

Decreasing fabrication geometries lowers costs. That's because chips are made on silicon wafers, and making things smaller means more chips fit on one wafer. Higher performance used to be achieved by

making the CPU faster, which meant increasing the clock speed. But faster machines required more power, which, combined with smaller geometries, produced more heat-generating power per unit of area. Processors hit the *power wall* around 2000 because the power density couldn't be increased without exceeding the melting point.

Salvation of sorts was found in the smaller fabrication geometries. The definition of CPU has changed; what we used to call a CPU is now called a *processor core*. *Multicore* processors are now commonplace. There are even systems, found primarily in data centers, with multiple multicore processors.

Microprocessors and Microcomputers

Another orthogonal architectural distinction is based on mechanical packaging. Figure 5-1 shows CPUs connected to memory and I/O. When the memory and I/O are not in the same physical package as the processor cores, we call it a *microprocessor*, whereas when everything is on a single chip, we use the term *microcomputer*. These are not really well-defined terms, and there is a lot of fuzziness around their usage. Some consider a microcomputer to be a computer system built around a microprocessor and use the term *microcontroller* to refer to what I've just defined as a microcomputer.

Microcomputers tend to be less powerful machines than microprocessors because things like on-chip memory take a lot of space. We're not going to focus on microcomputers much in this chapter because they don't have the same complexity of memory issues. However, once you learn to program, it is worthwhile to pick up something like an Arduino, which is a small Harvard architecture computer based on an Atmel AVR microcomputer chip. Arduinos are great for building all sorts of toys and blinky stuff.

To summarize: microprocessors are usually components of larger systems, while microcomputers are what you find in things like your dishwasher.

There's another variation called a *system on a chip (SoC)*. A passable but again fuzzy definition is that a SoC is a more complex microcomputer. Rather than having relatively simple on-chip I/O, a SoC can include things like Wi-Fi circuitry. SoCs are found in devices such as cell phones. There are even SoCs that include field-programmable gate arrays (FPGAs), which permit additional customization.